

Mawlana Bhashani Science and Technology University

Department of Information and Communication Technology

LAB REPORT

Lab Report No : 01

Lab Report On : Comparing Static and Non-Static Variables in Multithreading.

Course title : Software Engineering and Project Management Lab.

Course code : ICT 3108

Submitted by,

Name: Al-Shahariar.

ID: IT-24057

Year: 3rd

Semester: 1st

Dept. of ICT, MBSTU

Submitted to,

Teacher Name: Dr. Ziaur Rahman

Designation: Professor

Dept. of ICT, MBSTU

Date of Performance:

Date of Submission:

1. Problem Statement

In this lab I wrote a Java program with multiple threads to check how a static variable and a non-static (instance) variable behave when many threads update them at the same time. The program creates N threads, and each thread increases two counters K times.

The static counter belongs to the class, so there is only one copy and every thread works on it. The non-static counter is inside each object. Since every thread gets its own object, each thread only increases its own counter.

When all threads are done, I compare the two totals. I did this in two ways. In Experiment A the static counter is an `AtomicLong`, which is thread-safe. In Experiment B it is a normal `long` with no synchronization, so I could see what happens when updates get lost.

2. Objectives

The goal was to understand static vs non-static variables in practice, create and run many threads, calculate the absolute and percentage difference between two counts, see how the number of threads changes the result, and get a real feel for race conditions and thread safety.

3. Experimental Design and Formulas

Symbol	Meaning	Formula
N	Number of threads	-
K	Increments per thread	-
E	Expected count	$E = N \times K$
S	Final static counter	-
I	Sum of all non-static counters	I = sum of instanceCount of every thread
D	Absolute difference	$D = \lvert S - I \rvert$
P	Percentage difference	$P = \lvert S - I \rvert / I \times 100$ (0% if S = I = 0; undefined if only I = 0)

4. Program Design

As asked, everything is in one class, `Shahariar_Thread`, which implements `Runnable`. Each thread gets its own object of this class, so the object plays the role of the `CounterTask`.

The class has two static counters: `safeStatic` (an `AtomicLong`) and `unsafeStatic` (a plain `long`). It also has `instanceCount`, which is the non-static counter of each object. Inside `run()`, the third command-line argument decides which static counter gets increased.

In `main()` the program reads N, K and the mode, starts all N threads, and waits for each one with `join()`. After that it adds up all the instance counters and prints E, S, I, D and P.

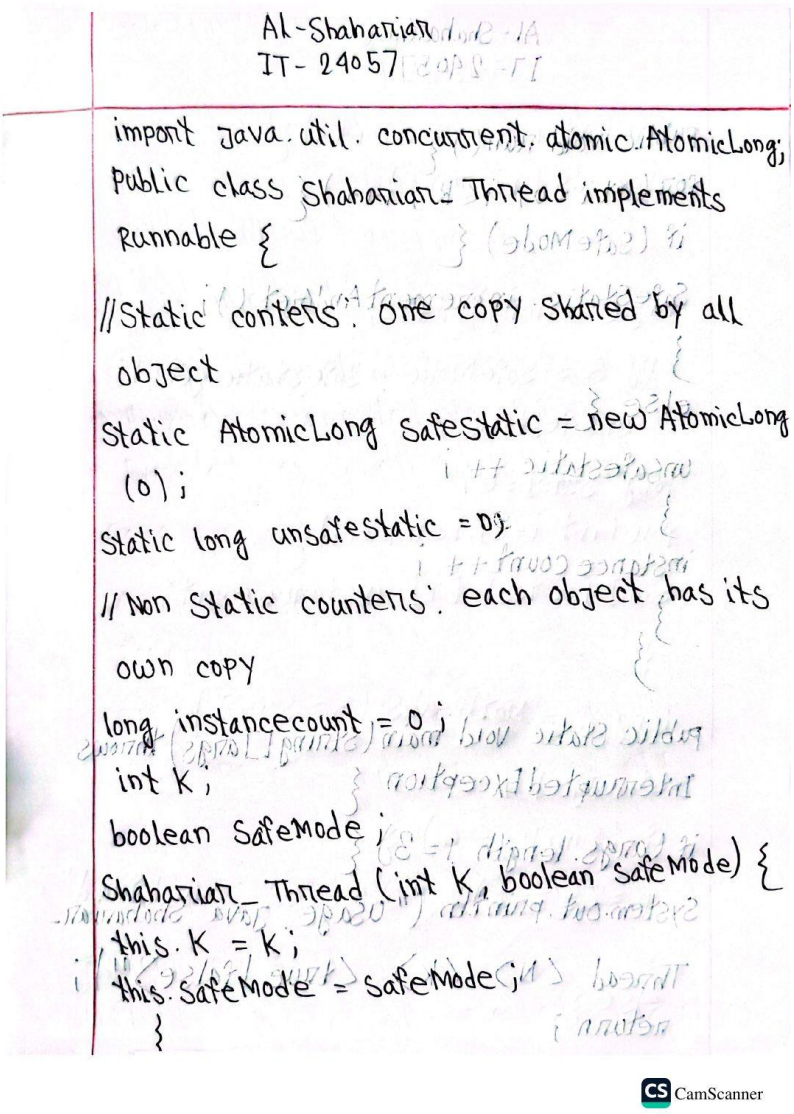
How to run:

```
javac Shahariar_Thread.java
java Shahariar_Thread 10 50000 true    // thread-safe
java Shahariar_Thread 10 50000 false   // unsynchronized
```

Machine used: Acer laptop, AMD Ryzen 5 7530U (6 cores, 12 threads, 2.00 GHz), 8 GB RAM, Windows 11 Home (26H2), Java SE 25 LTS.

5. Handwritten Code

The next pages are scans of my handwritten code for `Shahariar_Thread.java`. The typed version of the same code is in the `Lab_Thread_Counter` folder on GitHub.



Al-Shahariar
IT-24057

```
import java.util.concurrent.atomic.AtomicLong;
Public class Shahariar_Thread implements
Runnable {
//Static counters: one copy shared by all
object
Static AtomicLong SafeStatic = new AtomicLong
(0);
Static long unsafestatic = 0;
// Non Static counters: each object has its
own copy
long instancecount = 0;
int K;
boolean SafeMode;
Shahariar_Thread (int K, boolean SafeMode) {
this.K = K;
this.SafeMode = SafeMode;
}
```

CamScanner

Handwritten code, page 1 of 5

Al-Shahariar
IT-24057

```
Public void run() {
for (int i = 0; i < K; i++) {
 if (SafeMode) {
 SafeStatic.incrementAndGet();
 }
 else {
 unsafestatic ++ i
 }
 instance count++ ;
 }
 }
public Static void main (String[] args) throws
InterruptedException {
if (args.length != 3) {
System.out.println ("usage. Java Shahariar-
Thread <N> <K> <true|false>");
 return ;
```

CamScanner

Al-Shahariar
IT-24057

```
}
int n = Integer.parseInt(args[0]);
int K = Integer.parseInt(args[1]);
boolean SafeMode = Boolean.parseBoolean
(args[2]);
Shahariar_Thread[] task=new Shahariar_
Thread[n];
Thread[] thread = new Thread[n];
for(int i=0; i<n; i++){
tasks[i] = new Shahariar_Thread(K,
SafeMode);
thread[i] = new Thread(tasks[i]);
}
for(int i=0; i<n; i++){
threads[i].start();
}
```

CamScanner

Al-Shahariar
IT-24057

```
for (int i=0; i<n; i++){
threads[i].join();
}
long e =(long) n*k;
long S = SafeMode? Safestatic.get():
   unsafestatic;
long SumI=0;
for (int i=0; i<n; i++){
SumI += task[i].instancecount;
}
long d = Math.abs (S - SumI);
String P;
if (SumI == 0) {
P = (S==0)?"0.0000%.":"undefined";
}
else{
```

CamScanner

Al-Shahariar
IT-24057

```
P = String.format("%.4f%%", (d*100.0)/
  SumI);
}
System.out.println("Mode: "+(SafeMode?"
 true(AtomicLong, thread-Safe)": "false(long,
 unsynchronized)"));
System.out.println("Thread (N): "+n);
System.out.println("Increments(K): "+K);
System.out.println("Expected (E=N*K): "+e);
System.out.println("Static count (S):"+S);
System.out.println("Non-Static (I):"+SumI);
System.out.println("Abs difference (D):"+d);
System.out.println("Difference (P): "+P);
}
}
```

CamScanner

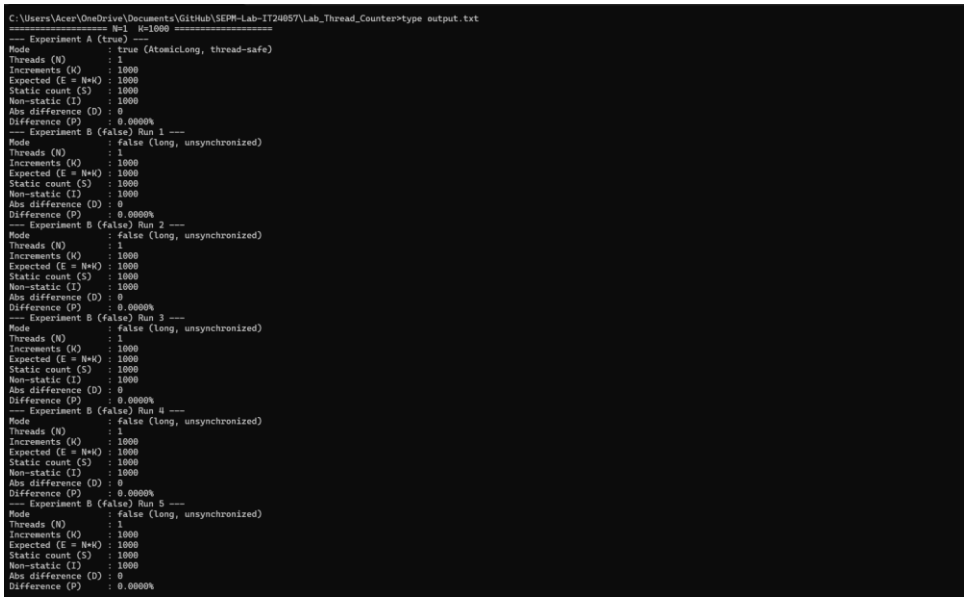
6. Test Inputs

I ran every test case once in thread-safe mode (`true`). Because the unsynchronized result changes every time, I ran that mode (`false`) five times for each case.

Test	Threads N	Increments K	Expected N × K	Command
TC1	1	1,000	1,000	java Shahariar_Thread 1 1000 true/false
TC2	2	10,000	20,000	java Shahariar_Thread 2 10000 true/false
TC3	5	10,000	50,000	java Shahariar_Thread 5 10000 true/false
TC4	10	50,000	500,000	java Shahariar_Thread 10 50000 true/false
TC5	20	50,000	1,000,000	java Shahariar_Thread 20 50000 true/false
TC6	50	50,000	2,500,000	java Shahariar_Thread 50 50000 true/false
TC7	100	50,000	5,000,000	java Shahariar_Thread 100 50000 true/false

7. Program Output (Screenshots)

I used `run_all.bat` to run all the cases in one go and saved the output to `output.txt`. The screenshots below come from that file, and the full output is also on GitHub.



TC1: N = 1, K = 1,000

```
================== N=2  K=10000 ==================
--- Experiment A (true) ---
Mode               : true (AtomicLong, thread-safe)
Threads (N)        : 2
Increments (K)     : 10000
Expected (E = N*K) : 20000
Static count (S)   : 20000
Non-static (I)     : 20000
Abs difference (D) : 0
Difference (P)     : 0.0000%
--- Experiment B (false) Run 1 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 2
Increments (K)     : 10000
Expected (E = N*K) : 20000
Static count (S)   : 16971
Non-static (I)     : 20000
Abs difference (D) : 3029
Difference (P)     : 15.1450%
--- Experiment B (false) Run 2 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 2
Increments (K)     : 10000
Expected (E = N*K) : 20000
Static count (S)   : 16756
Non-static (I)     : 20000
Abs difference (D) : 3244
Difference (P)     : 16.2200%
--- Experiment B (false) Run 3 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 2
Increments (K)     : 10000
Expected (E = N*K) : 20000
Static count (S)   : 18646
Non-static (I)     : 20000
Abs difference (D) : 1354
Difference (P)     : 6.7700%
--- Experiment B (false) Run 4 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 2
Increments (K)     : 10000
Expected (E = N*K) : 20000
Static count (S)   : 17183
Non-static (I)     : 20000
Abs difference (D) : 2817
Difference (P)     : 14.0850%
--- Experiment B (false) Run 5 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 2
Increments (K)     : 10000
Expected (E = N*K) : 20000
Static count (S)   : 14686
Non-static (I)     : 20000
Abs difference (D) : 5314
Difference (P)     : 26.5700%
```

TC2: N = 2, K = 10,000

```
================== N=5  K=10000 ==================
--- Experiment A (true) ---
Mode               : true (AtomicLong, thread-safe)
Threads (N)        : 5
Increments (K)     : 10000
Expected (E = N*K) : 50000
Static count (S)   : 50000
Non-static (I)     : 50000
Abs difference (D) : 0
Difference (P)     : 0.0000%
--- Experiment B (false) Run 1 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 5
Increments (K)     : 10000
Expected (E = N*K) : 50000
Static count (S)   : 20907
Non-static (I)     : 50000
Abs difference (D) : 29093
Difference (P)     : 58.1860%
--- Experiment B (false) Run 2 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 5
Increments (K)     : 10000
Expected (E = N*K) : 50000
Static count (S)   : 19194
Non-static (I)     : 50000
Abs difference (D) : 30806
Difference (P)     : 61.6120%
--- Experiment B (false) Run 3 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 5
Increments (K)     : 10000
Expected (E = N*K) : 50000
Static count (S)   : 15313
Non-static (I)     : 50000
Abs difference (D) : 34687
Difference (P)     : 69.3740%
--- Experiment B (false) Run 4 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 5
Increments (K)     : 10000
Expected (E = N*K) : 50000
Static count (S)   : 18152
Non-static (I)     : 50000
Abs difference (D) : 31848
Difference (P)     : 63.6960%
--- Experiment B (false) Run 5 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 5
Increments (K)     : 10000
Expected (E = N*K) : 50000
Static count (S)   : 20682
Non-static (I)     : 50000
Abs difference (D) : 29318
Difference (P)     : 58.6360%
```

TC3: N = 5, K = 10,000

```
================== N=10  K=50000 ==================
--- Experiment A (true) ---
Mode               : true (AtomicLong, thread-safe)
Threads (N)        : 10
Increments (K)     : 50000
Expected (E = N*K) : 500000
Static count (S)   : 500000
Non-static (I)     : 500000
Abs difference (D) : 0
Difference (P)     : 0.0000%
--- Experiment B (false) Run 1 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 10
Increments (K)     : 50000
Expected (E = N*K) : 500000
Static count (S)   : 71833
Non-static (I)     : 500000
Abs difference (D) : 428167
Difference (P)     : 85.6334%
--- Experiment B (false) Run 2 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 10
Increments (K)     : 50000
Expected (E = N*K) : 500000
Static count (S)   : 81617
Non-static (I)     : 500000
Abs difference (D) : 418383
Difference (P)     : 83.6766%
--- Experiment B (false) Run 3 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 10
Increments (K)     : 50000
Expected (E = N*K) : 500000
Static count (S)   : 68011
Non-static (I)     : 500000
Abs difference (D) : 431989
Difference (P)     : 86.3978%
--- Experiment B (false) Run 4 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 10
Increments (K)     : 50000
Expected (E = N*K) : 500000
Static count (S)   : 68515
Non-static (I)     : 500000
Abs difference (D) : 431485
Difference (P)     : 86.2970%
--- Experiment B (false) Run 5 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 10
Increments (K)     : 50000
Expected (E = N*K) : 500000
Static count (S)   : 72578
Non-static (I)     : 500000
Abs difference (D) : 427422
Difference (P)     : 85.4844%
```

TC4: N = 10, K = 50,000

```
================== N=20  K=50000 ==================
--- Experiment A (true) ---
Mode               : true (AtomicLong, thread-safe)
Threads (N)        : 20
Increments (K)     : 50000
Expected (E = N*K) : 1000000
Static count (S)   : 1000000
Non-static (I)     : 1000000
Abs difference (D) : 0
Difference (P)     : 0.0000%
--- Experiment B (false) Run 1 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 20
Increments (K)     : 50000
Expected (E = N*K) : 1000000
Static count (S)   : 153777
Non-static (I)     : 1000000
Abs difference (D) : 846223
Difference (P)     : 84.6223%
--- Experiment B (false) Run 2 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 20
Increments (K)     : 50000
Expected (E = N*K) : 1000000
Static count (S)   : 168596
Non-static (I)     : 1000000
Abs difference (D) : 831404
Difference (P)     : 83.1404%
--- Experiment B (false) Run 3 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 20
Increments (K)     : 50000
Expected (E = N*K) : 1000000
Static count (S)   : 141009
Non-static (I)     : 1000000
Abs difference (D) : 858991
Difference (P)     : 85.8991%
--- Experiment B (false) Run 4 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 20
Increments (K)     : 50000
Expected (E = N*K) : 1000000
Static count (S)   : 90331
Non-static (I)     : 1000000
Abs difference (D) : 909669
Difference (P)     : 90.9669%
--- Experiment B (false) Run 5 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 20
Increments (K)     : 50000
Expected (E = N*K) : 1000000
Static count (S)   : 159176
Non-static (I)     : 1000000
Abs difference (D) : 840824
Difference (P)     : 84.0824%
```

TC5: N = 20, K = 50,000

```
================== N=50  K=50000 ==================
--- Experiment A (true) ---
Mode               : true (AtomicLong, thread-safe)
Threads (N)        : 50
Increments (K)     : 50000
Expected (E = N*K) : 2500000
Static count (S)   : 2500000
Non-static (I)     : 2500000
Abs difference (D) : 0
Difference (P)     : 0.0000%
--- Experiment B (false) Run 1 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 50
Increments (K)     : 50000
Expected (E = N*K) : 2500000
Static count (S)   : 705230
Non-static (I)     : 2500000
Abs difference (D) : 1794770
Difference (P)     : 71.7908%
--- Experiment B (false) Run 2 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 50
Increments (K)     : 50000
Expected (E = N*K) : 2500000
Static count (S)   : 918087
Non-static (I)     : 2500000
Abs difference (D) : 1581913
Difference (P)     : 63.2765%
--- Experiment B (false) Run 3 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 50
Increments (K)     : 50000
Expected (E = N*K) : 2500000
Static count (S)   : 466348
Non-static (I)     : 2500000
Abs difference (D) : 2033652
Difference (P)     : 81.3461%
--- Experiment B (false) Run 4 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 50
Increments (K)     : 50000
Expected (E = N*K) : 2500000
Static count (S)   : 515668
Non-static (I)     : 2500000
Abs difference (D) : 1984332
Difference (P)     : 79.3733%
--- Experiment B (false) Run 5 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 50
Increments (K)     : 50000
Expected (E = N*K) : 2500000
Static count (S)   : 950834
Non-static (I)     : 2500000
Abs difference (D) : 1549166
Difference (P)     : 61.9666%
```

TC6: N = 50, K = 50,000

```
================== N=100  K=50000 ==================
--- Experiment A (true) ---
Mode               : true (AtomicLong, thread-safe)
Threads (N)        : 100
Increments (K)     : 50000
Expected (E = N*K) : 5000000
Static count (S)   : 5000000
Non-static (I)     : 5000000
Abs difference (D) : 0
Difference (P)     : 0.0000%
--- Experiment B (false) Run 1 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 100
Increments (K)     : 50000
Expected (E = N*K) : 5000000
Static count (S)   : 2855537
Non-static (I)     : 5000000
Abs difference (D) : 2144463
Difference (P)     : 42.8893%
--- Experiment B (false) Run 2 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 100
Increments (K)     : 50000
Expected (E = N*K) : 5000000
Static count (S)   : 3032145
Non-static (I)     : 5000000
Abs difference (D) : 1967855
Difference (P)     : 39.3571%
--- Experiment B (false) Run 3 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 100
Increments (K)     : 50000
Expected (E = N*K) : 5000000
Static count (S)   : 1791964
Non-static (I)     : 5000000
Abs difference (D) : 3208036
Difference (P)     : 64.1607%
--- Experiment B (false) Run 4 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 100
Increments (K)     : 50000
Expected (E = N*K) : 5000000
Static count (S)   : 1910899
Non-static (I)     : 5000000
Abs difference (D) : 3089101
Difference (P)     : 61.7820%
--- Experiment B (false) Run 5 ---
Mode               : false (long, unsynchronized)
Threads (N)        : 100
Increments (K)     : 50000
Expected (E = N*K) : 5000000
Static count (S)   : 3328159
Non-static (I)     : 5000000
Abs difference (D) : 1671841
Difference (P)     : 33.4368%
```

TC7: N = 100, K = 50,000

8. Result Analysis

8.1 Experiment A – Thread-safe (AtomicLong)

Threads	Expected	Static count (safe)	Non-static total	Abs. difference	Difference (%)
1	1,000	1,000	1,000	0	0%
2	20,000	20,000	20,000	0	0%
5	50,000	50,000	50,000	0	0%
10	500,000	500,000	500,000	0	0%
20	1,000,000	1,000,000	1,000,000	0	0%
50	2,500,000	2,500,000	2,500,000	0	0%
100	5,000,000	5,000,000	5,000,000	0	0%

8.2 Experiment B – Unsynchronized (plain long), 5 runs each

Threads	Expected	Run 1 S	Run 2 S	Run 3 S	Run 4 S	Run 5 S	Non-static total	Avg. D	Avg. P (%)
1	1,000	1,000	1,000	1,000	1,000	1,000	1,000	0	0.0000
2	20,000	16,971	16,756	18,646	17,183	14,686	20,000	3,151.6	15.7580
5	50,000	20,907	19,194	15,313	18,152	20,682	50,000	31,150.4	62.3008
10	500,000	71,833	81,617	68,011	68,515	72,578	500,000	427,489.2	85.4978
20	1,000,000	153,777	168,596	141,009	90,331	159,176	1,000,000	857,422.2	85.7422
50	2,500,000	705,230	918,087	466,348	515,668	950,834	2,500,000	1,788,766.6	71.5507
100	5,000,000	2,855,537	3,032,145	1,791,964	1,910,899	3,328,159	5,000,000	2,416,259.2	48.3252

Avg. P = (P1 + P2 + P3 + P4 + P5) / 5, where each P = |S − I| / I × 100.

8.3 Range of P across the 5 runs

Threads	Min P (%)	Max P (%)	Spread (points)
1	0	0	0
2	6.7700	26.5700	19.80
5	58.1860	69.3740	11.19
10	83.6766	86.3978	2.72
20	83.1404	90.9669	7.83
50	61.9666	81.3461	19.38
100	33.4368	64.1607	30.72

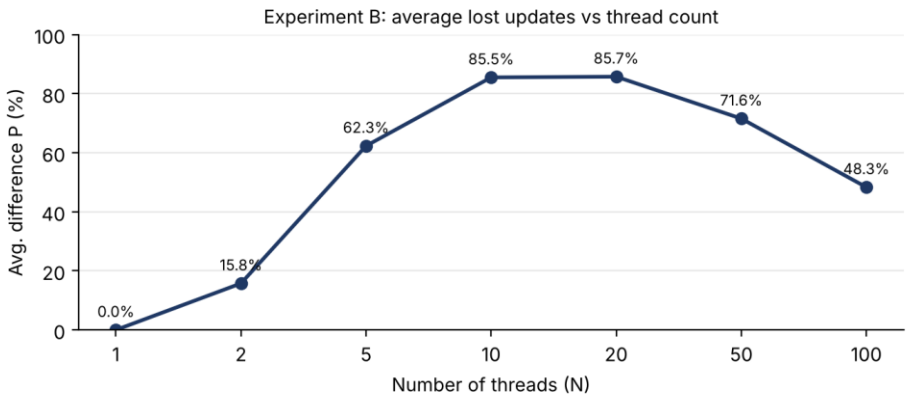


Figure 1: Average percentage difference in Experiment B. Experiment A stayed at 0% for every N.

9. Observation

The thread-safe run behaved exactly as expected. In all seven cases S, I and E were equal, so the difference was 0%. `AtomicLong` did not lose a single update.

The unsynchronized run was more interesting. With only one thread, S was always 1,000, which makes sense because there was no other thread to clash with. As soon as I used two threads, the static count came out lower than expected in every run. The non-static total, on the other hand, was always exactly N × K, since each thread had its own counter.

The numbers also changed a lot from run to run, even with the same input. With 2 threads the loss went from 6.77% in one run to 26.57% in another. With 100 threads it went from 33.44% to 64.16%. The most stable case was 10 threads, where all five runs stayed between 83.68% and 86.40%.

I expected the loss to keep growing with more threads, but that is not what happened. The average went up from 0% (1 thread) to 15.76% (2), 62.30% (5), and about 85.5% at both 10 and 20 threads. Then it came down to 71.55% at 50 threads and 48.33% at 100 threads.

I think the drop happens because my CPU (Ryzen 5 7530U) can only run 12 threads truly at the same time. With 50 or 100 threads, most of them are waiting for their turn, and the first threads can finish a big part of their loop before the last ones even start. So fewer threads actually overlap, and fewer increments get lost in percentage terms.

10. Analysis Questions

1. What is the difference between a static variable and a non-static variable in Java?

A static variable belongs to the class itself, so there is only one copy no matter how many objects we create. A non-static (instance) variable belongs to an object, and every object gets its own separate copy.

2. Why do all threads share the same static counter?

Because the static counter is stored once for the whole class. All the thread objects are of the same class, so they all read and write that one variable.

3. Why does each thread have its own non-static counter in this experiment?

In my program I create a new Shahariar_Thread object for every thread. instanceCount is inside that object, so each thread only touches its own copy and no other thread can change it.

4. Why is join() required before calculating the final counts?

join() makes main() wait until a thread has finished. If I skip it, main() might read the counters while some threads are still running, and the totals would be wrong. join() also makes sure the values written by the thread are visible to main().

5. Why can the unsynchronized static count be lower than the expected count?

count++ looks like one step but it is really three: read the value, add 1, and write it back. If two threads read the same value at the same time, both write back the same number, and one increment is lost. This is a race condition.

6. Does increasing the number of threads always increase the percentage difference?

No. In my results the average loss went up to about 85% at 10 and 20 threads, but then dropped to 71.55% at 50 threads and 48.33% at 100 threads. More threads do give more chances for a race, but the real result depends on how many threads run at the same moment. My CPU can only run 12 at once, so with 50 or 100 threads many of them just wait, and the overlap actually goes down.

7. Why might two runs with the same number of threads produce different results?

The operating system decides when each thread runs, and this is different every time. So the exact moments when two threads collide change from run to run, and the number of lost updates changes too. Other programs running in the background and the JIT compiler also affect the timing.

8. What changes when AtomicLong is used instead of a regular long?

With AtomicLong, incrementAndGet() does the read, add and write as one single step that cannot be interrupted (it uses the CPU's compare-and-swap). So no update is ever lost and S always equals E. It is a bit slower than a plain long, but the result is always correct.

9. How would you modify the program so that all threads share one instance counter as well as the static counter?

I would create just one object and give that same object to all the threads, for example Shahariar_Thread shared = new Shahariar_Thread(k, true); and then new Thread(shared) for every thread. Now the instance counter is shared too, so it also needs to be safe: I would change it to an AtomicLong or put the increment inside a synchronized block.

11. Conclusion

The main thing I learned is that static or non-static only tells us who owns a variable. It does not make the variable thread-safe. A static variable is shared by every thread, so if many threads change it at the same time, we still need `AtomicLong` or `synchronized`. The instance counters were always correct only because no other thread could touch them.

The unsynchronized experiment lost updates every time more than one thread ran, and the amount changed from run to run. The loss also did not simply grow with the number of threads. It peaked at around 85% for 10 to 20 threads and then fell to about 48% at 100 threads, which shows that it depends on scheduling and the CPU. The thread-safe experiment worked as the control: S and I matched in every test and the difference stayed at 0%.